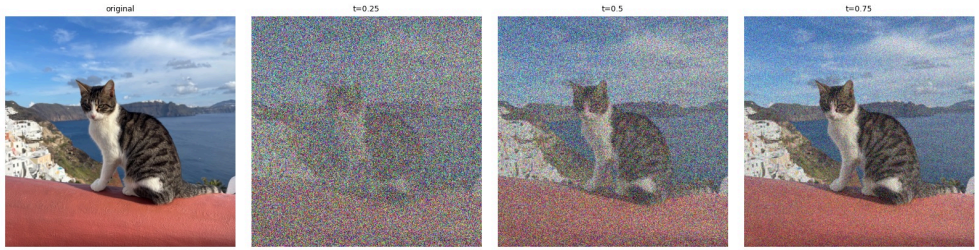
Project 3: Fun with Flow Models



A noisy cat in Santorini, Greece.

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

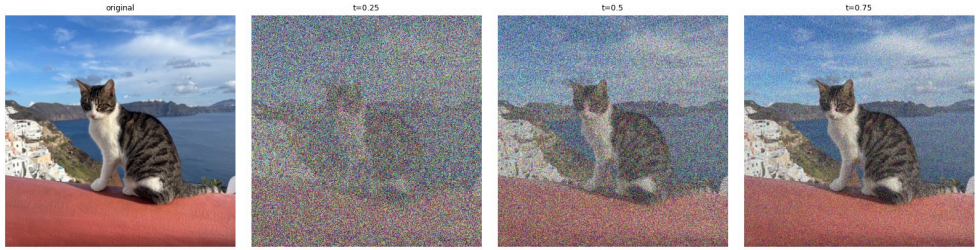
Part A.1: Visualizing Training and Sampling

The optimal-transport formulation of flow matching uses linear interpolation to construct an intermediate image xt in between the clean image x1 and an image of pure Gaussian noise x0. t = 0 corresponds to pure noise, and t = 1 corresponds to a clean image.

$$x_t = (1-t)\cdot x_0 + t\cdot x_1 \quad t\in[0,1], \quad x_0 = \epsilon \sim N(0, I)$$

1.1 Constructing Noisy Training Inputs

First, I sampled a tensor with shape (H, W, 3) from a Guassian distribution. Then, I computed a weighted sum of the original image and the noisy image at t = [0.25, 0.50, and 0.75]. The bigger the t, the more of the original image information is retained!

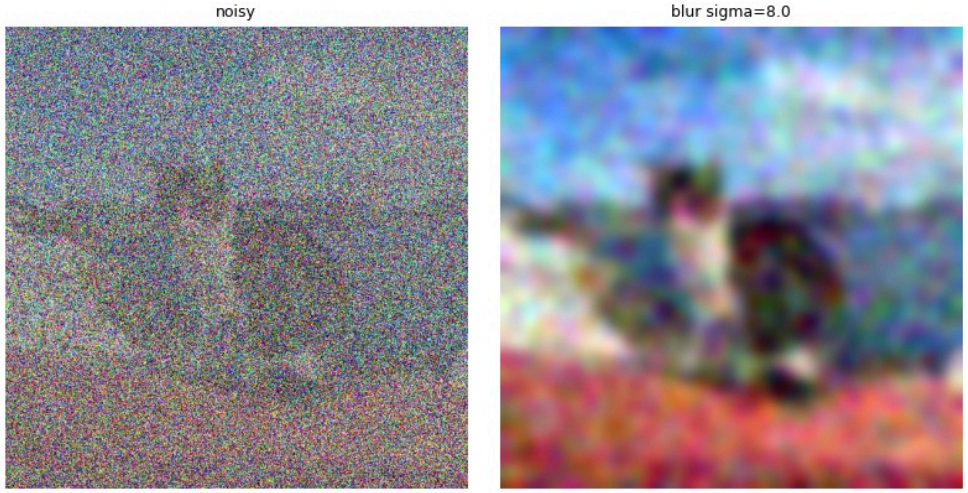


The original santorini_cat.jpg image and interpolations at t = [0.25, 0.50, 0.75]

1.2 Classical Denoising

Applying a Gaussian blur to an image essentially averages neighboring pixels. A strong blur reduces noise but also removes image detail. In this part, I applied Gaussian blurs (with sigma scaled based on t) to reconstruct the original image. This provides a helpful baseline to compare with learned flow models later on!

Below, you can view each of the 3 noisy versions of santorini_cat.jpg side-by-side with a Gaussian-denoised version.



Noisy image at t = 0.25 and Gaussian denoising with sigma = 8.0

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model



Noisy image at t = 0.50 and Gaussian denoising with sigma = 4.0



Noisy image at t = 0.75 and Gaussian denoising with sigma = 2.0

Contents

- Part A.1: Visualizing Training and Sampling
- Part A.2: Conditional Generation and Guidance
- Part A.3: Creative Applications
- Part B.1: Training a Single-Step Denoising UNet
- Part B.2: Training a Flow Matching Model

1.3 One-Step Denoising

Next, I used a pretrained flow model PixNerd-XXL-P16-T2Ito predict the velocity given a noisy image and the timestep.

Since PixNerd was trained with text-conditioning, so I used the placeholder "a high quality photo" as the text input for now.

I implemented one-step denoising by extraploating directly to the clean endpoint with this single, predicted velocity:

$$\hat{x_1} = x_t + (1 - t) \cdot v_\theta(x_t, t, c)$$

These are the results of one-step denoising! The leftmost image is the original, the middle image is the noisy interpolation, and the rightmost image is the predicted clean image using one-step denoising.



Noisy image at t = 0.25 and an estimate of the original image with one-step denoising



Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

Noisy image at t = 0.50 and an estimate of the original image with one-step denoising



Noisy image at t = 0.75 and an estimate of the original image with one-step denoising

1.4 Visualizing Velocity

For each time t = 0.25, 0.50, 0.75, I visualized the noisy image x_t, the predicted one-step update (1 - t) v, and the ground truth update x_1 - x_t. I also included an error heatmap based on the root-mean-square difference between predicted update and the ground-truth update. First, I squared the differences at each pixel in each color channels, averaged the squared differences across the RGB channels, then took the square root. This results in one error value per pixel position.

Contents

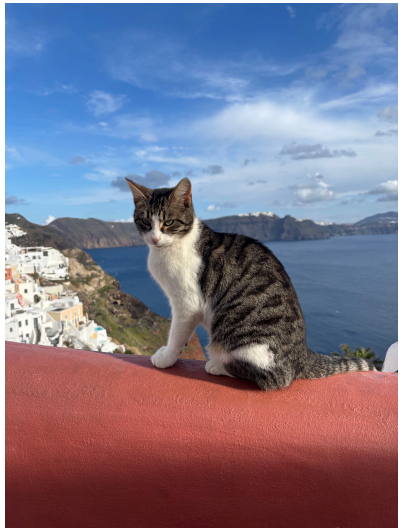
Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

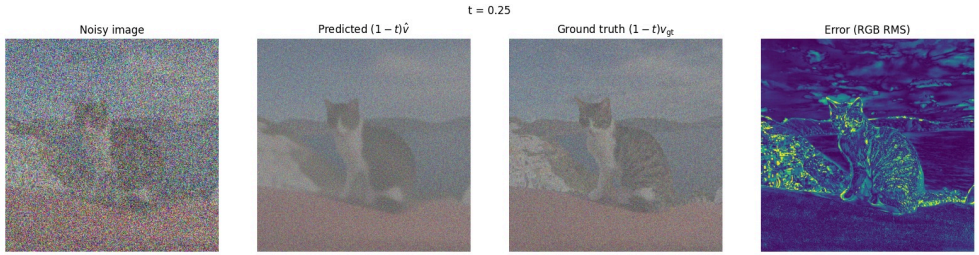
Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

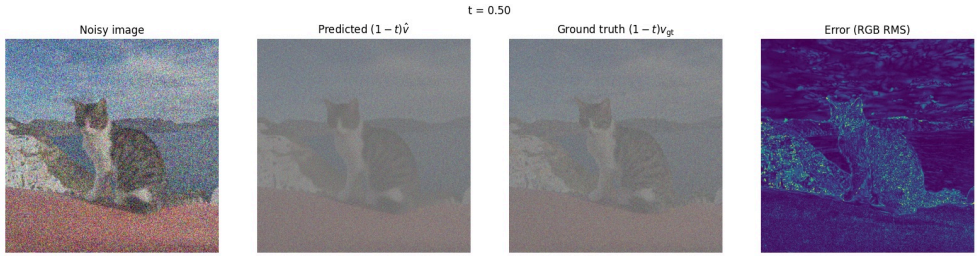
Part B.2: Training a Flow Matching Model



The original santorini_cat.jpg image



Visualizing velocities and RGB RMS error at t = 0.25



Visualizing velocities and RGB RMS error at t = 0.50

Contents

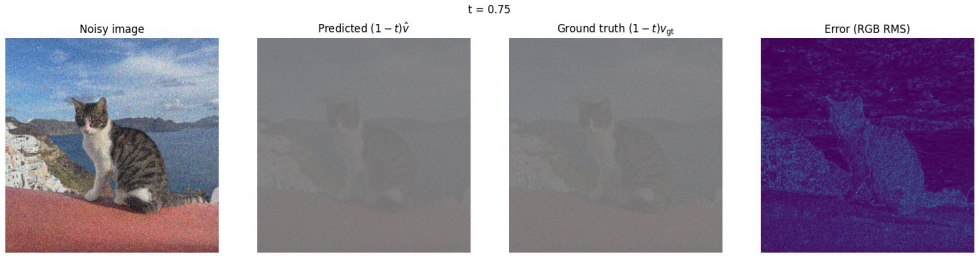
Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model



Visualizing velocities and RGB RMS error at t = 0.75

1.5 Iterative Denoising with Euler Sampling

Rather than denoising the image from x_0 (pure noise) to x_1 (clean image) in a single step, we can use multi-step sampling. I implemented Euler integration by dividing the time from [0, 1] into small step sizes. At each timestep, I used the flow-matching model to predict the velocity at the current timestep, then took a small step in the direction of the predicted veolocity.

$$x_{t+\Delta t} = x_t + \Delta t \cdot v_\theta(x_t, t, c)$$

Here are the results of starting with a noisy version of santorini_cat.jpg at t = 0.50, and taking 25 Euler integration steps.

Contents

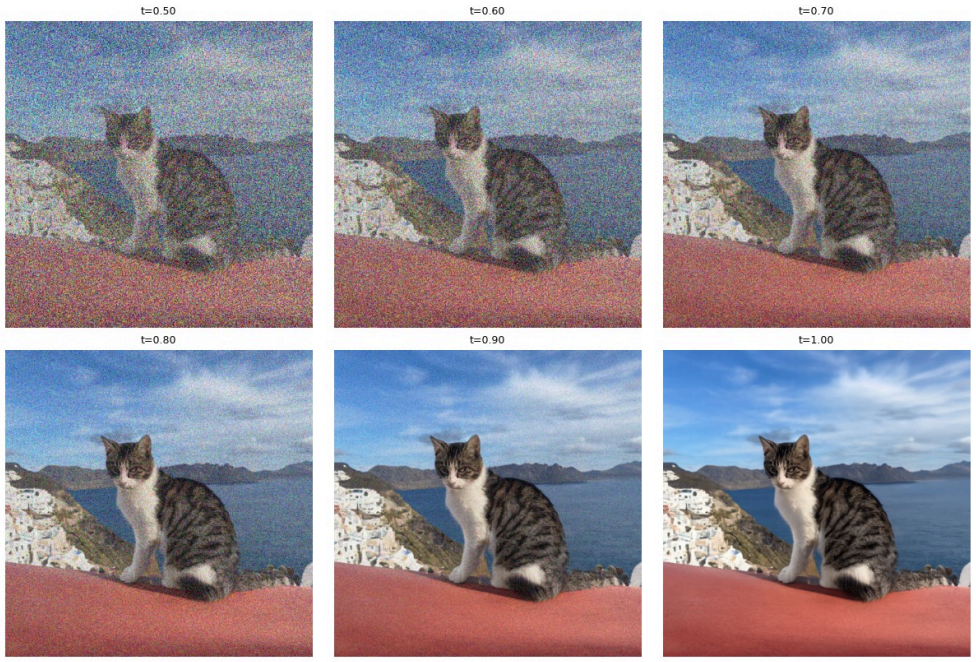
Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model



Multi-step iterative Euler denoising, starting from t = 0.50 and taking 25 steps to reach t = 1.0. These are the intermediate results at every 5 steps. The initial state is t = 0.50 and the final image is t = 1.0.

Comparing the results of multi-step denoising with the original image (the ground truth!), Gaussian-blur denoising, and one-step denoising. Out of the 3 denoising methods so far, multi-step denoising recovers the original image the best. In multi-step denoising, at every step the model can base its prediction of the velocity on a less noisy input x_t; therefore every step the algorithm takes can "self-adjust" to more closely follow the true flow path.



Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

Comparing the original image and 3 different denoising strategies: Gaussian blur, one-step denoising, and multi-step denoising.

1.6 Generating Images from Noise

By passing in pure noise (x_0) and t = 0 to the multi-step denoising algorithm, we can generate images from scratch!

Here are 8 results of denoising pure noise, with the prompt "a high quality photo". All 8 results were of human-like subjects (quite uncanny valley). It seems that PixNerd favors human subjects when given this generic prompt!



Generating 8 images by starting with pure noise initialized with different seeds.

Part A.2: Conditional Generation and Guidance

2.1 Classifier-Free Guidance (CFG)

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

Conditional guidance is when a model generates an image based on a certain condition, such as a provided text prompt. In classifier-free guidance (CFG), the flow-matching model is trained to denoise images with a condition, and without any condition (e.g. with a "null" or empty-string as the condition). Then, during each step of sampling, we run a forward pass through the model with the condition to produce a conditional velocity v_c, and another forward pass through the same model without any condition to produce an unconditional velocity v_u. Subtracting v_c - v_u gives us a vector that points from the unconditional flow to the conditional flow (i.e. a direction pointing towards the class/condition). Adding this vector to the conditional velocity (with a scalar weight to control how much we wish to "exaggerate" the guidance) results in a velocity that is guided towards the desired condition.

Setting w greater than 1 pushes the flow-matching model to generate features or images that emphasize what make the given condition distinct.

This algorithm is called "classifier-free" because it doesn't involve training a separate classifier to score whether or not the model produces the relevant image for the given condition.

$$v_{cfg} = v_c + w \cdot (v_c - v_u)$$
$$x_{t+\Delta t} = x_t + \Delta t \cdot v_{cfg}$$

Here are the results of generating images from pure-noise using the same text prompt "a high quality photo" as in part 1.6, but now implemented with CFG with w=7. Using CFG, the images are still of human subjects but look much more realistic. Using a high value for w helped push the generated images closer towards realistic subjects.

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model



Generating 8 images with the text prompt "a high quality photo" via classifier-free guidance (w=7).

Part A.3: Creative Applications

3.1 Image-to-Image Editing

Denoising also allows us to "edit" pictures! In order to denoise an image, the flow model has to be "creative" in order to guide the noisy image back onto thee manifold of natural images.

If we take an image, add some noise to it, then run multi-step denoising from a partially noisy image, we can get a result that is similar to the original image but with some creative, artistic edits!

Below, I ran the forward, noise-adding process to an image of the Campanile to get a noisy image at various times: t = [0.02, 0.04, 0.06, 0.10, 0.14, 0.25, 0.5]. Then, I ran the denoising process starting from these timestamps (again, with the generic text prompt "a high quality photo" passed to the PixNerd flow model) to "edit" the pictures. At early starting times, the edited final result is mostly unrelated to the original

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

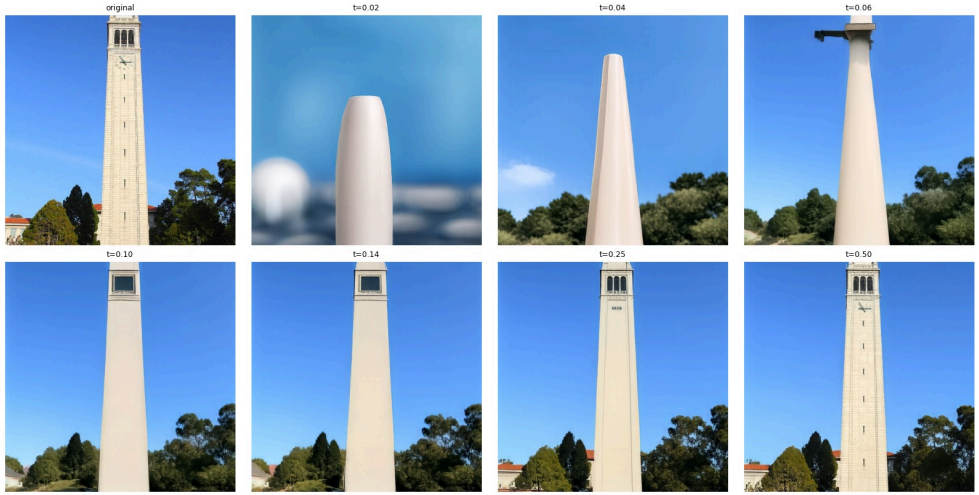
Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

image; they often depict a human subject, reflecting this pretrained PixNerd model's tendency to produce human subjects when given the prompt "a high quality photo." At later starting times, more of the original image's info is retained in the input to the model. So the final result looks more related to the original clean image.

I also tried editing 2 of my own images: a picture of a cat I took on Santorini island, Greece and a picture of a windmill I took in the Netherlands.



The results of editing campanile.jpg starting at various intial times. Earlier starting times starting denoising from almost pure noise and resulted in very different final images than the original campanile.jpg.

Contents

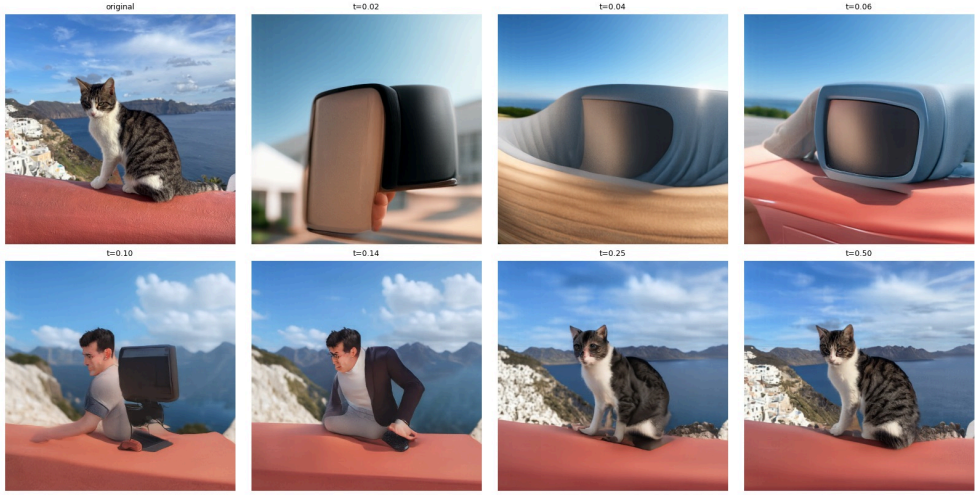
Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model



The results of editing a cat on the Greek island of Santorini starting at various intial times.



The results of editing an image of a windmill from the Netherlands starting at various intial times.

Editing Hand-Drawn and Web Images

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

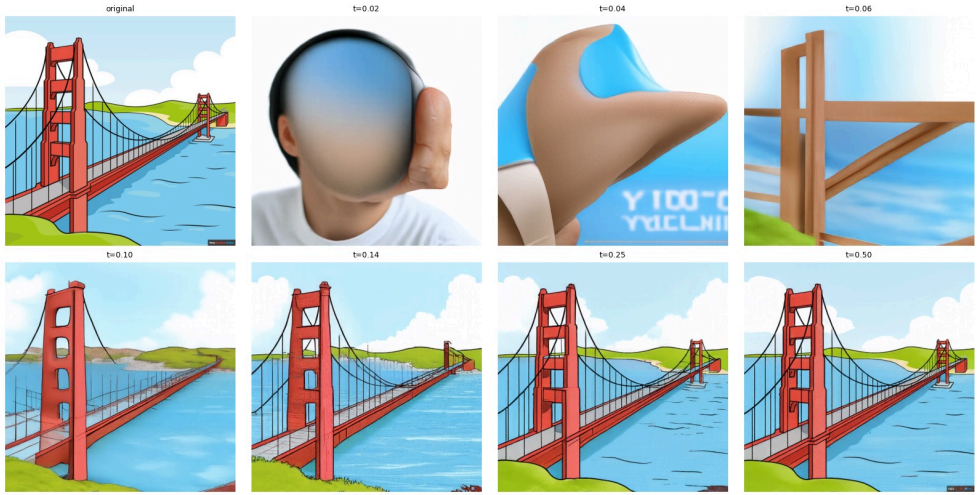
Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

Editing via denoising can produce some fun results if we start out with a non-realistic image, like drawings or pixel art, then try to denoise it back onto the natural image manifold.

Here are the results of editing-via-denoising on a web image of the Golden Gate Bridge, a custom-drawn image of a T-rex surfing a wave, and a custom-drawn image of a hotdog. I tried editing starting from 7 different times t = [0.02, 0.04, 0.06, 0.10, 0.14, 0.25, 0.5]. At early start times such as 0.02 and 0.04, the final denoised result often looks very different from the original input. For the Golden Gate Bridge drawing, the model's strong preference for generating humanoid subject overrides any information about the original drawing, besides a patch of blue from the water. Interestingly, for the surfing T-rex, the model adds a small logo or watermark design in the bottom right corner up until t = 0.25 (and also hallucinates text). For the hotdog drawing, the model denoises the red sausage into a roughly rectangular or cylindrical shape, but isn't able to recover an identifiable hotdog until the start time reaches about 0.14 or 0.25.



Editing a drawing of the Golden Gate Bridge via denoising

Contents

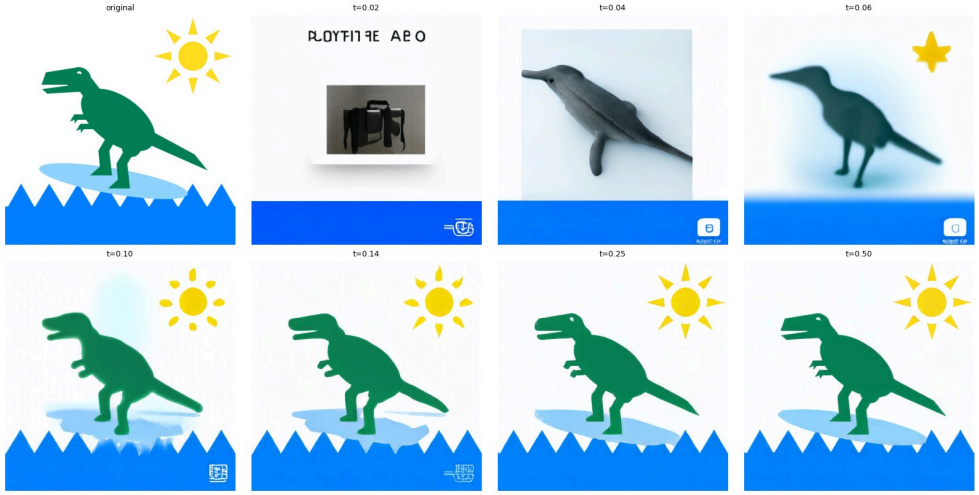
Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model



Editing a drawing I made of a T-rex hanging ten via denoising



Editing a drawing I made of a hotdog via denoising

3.2 Text-Guided Image Editing

Next, I tried editing via denoising with a prompt describing the edit procedure.

The text-guided image editing results more closely reflected the text prompt when

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

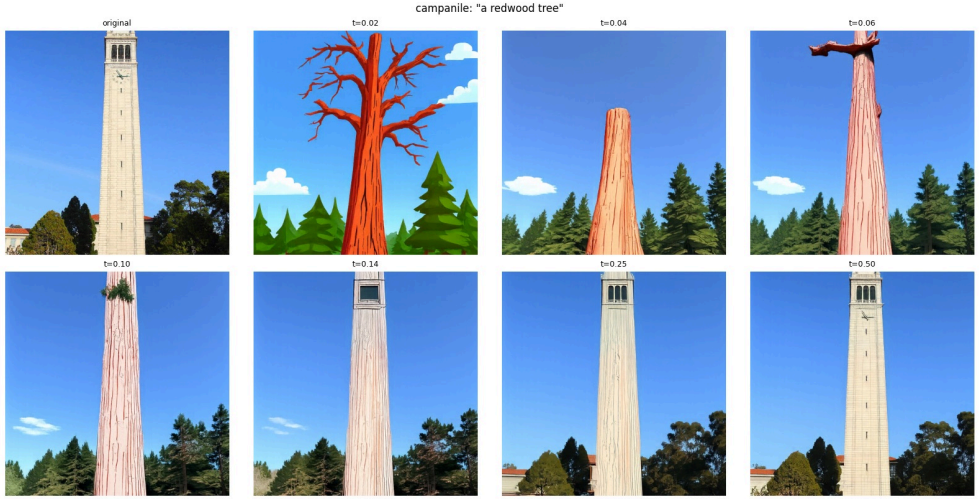
Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

starting from early time steps like t = 0.02, t = 0.04, or t = 0.06. When the editing/denoising process began from later times, the flow model was not able to edit away from the original image information still kept in the partially noisy starting image.

So starting the editing process earlier on (where the initial input is closer to pure noise) generates a final results that favors the text prompt, whereas starting the editing process later (where the initial input still retains a lot of the original image's information) generates a final result that is mostly the clean image.



Editing campanile.jpg with the text prompt "a redwod tree"

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model



Editing santorini_cat.jpg with the text prompt "a dragon sitting on top of a pile of treasure"



Editing windmill.jpg with the text prompt "an ice cream sundae"

3.3 Visual Anagrams

Visual anagrams are images that look like one thing when right-side up, but look like another thing when flipped upside down!

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

At every time step, I denoised the current x_t normally with prompt p_1 to produce a velocity estimate v_1. Next, I rotated the current x_t by 180 degrees, denoised it with the second prompt p_2, and obtained a velocity estimate v_2. Then, I flipped v_2 back around (since the model produces a velocity estimate for an image, the v_2 has the same dimensions as the image), and averaged the two velocity estimates. Finally, I performed an Euler step with the average of the two velocities.

$$v_1 = \text{CFG velocity}(x_t, t, p_1)$$
$$v_2 = \text{flip}(\text{CFG velocity}(\text{flip}(x_t), t, p_2))$$
$$v = \frac{v_1 + v_2}{2}$$



A visual anagram I made inspired by The Odyssey: a wooden horse and an owl representing Athena!

Contents

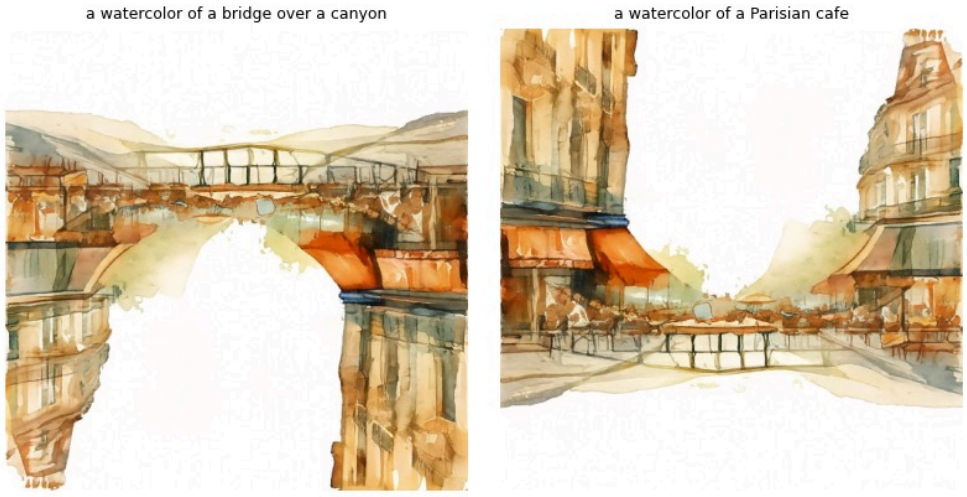
Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model



A visual anagram of a bridge over a canyon and a Parisian cafe.

3.4 Hybrid Images

Hybrid images are images that appear to depict different subjects when viewed up close and from far away. They take advantage of how our eyes tend to perceive large features (i.e. low frequencies) when viewing something from far away, but are more sensitive to fine details and edges (i.e. high frequencies) when viewing things from up close.

To create hybrid images with flow models, at every denoising timestep t, I used the flow model to predict two velocities given the same x_t but different prompts. Then, I combined the low frequencies of one velocity estimate with the high frequencies of another velocity estimate.

A low-pass filter invovles convolving the velocity estimate with a Gaussian kernel. The high-pass filter is the predition, minus a low-pass version of itself.

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

$$
\begin{aligned}
v_1 &= \text{CFG velocity}(x_t, t, p_1) \\
v_2 &= \text{CFG velocity}(x_t, t, p_2) \\
v &= f_{lowpass}(v1) + f_{highpass}
\end{aligned}
$$

These are 2 hybrid images I made. The first one is a hybrid of a volcano and a wizard! The second one is a spiderman :)



A hybrid image that looks like "a volcano" from up close (zoomed in)...



...and "a wizard" from afar (zoomed far away).

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model



A hybrid image that looks like "a man walking through New York City" from up close (zoomed in)...



...and "a spider" from afar (zoomed far away).

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

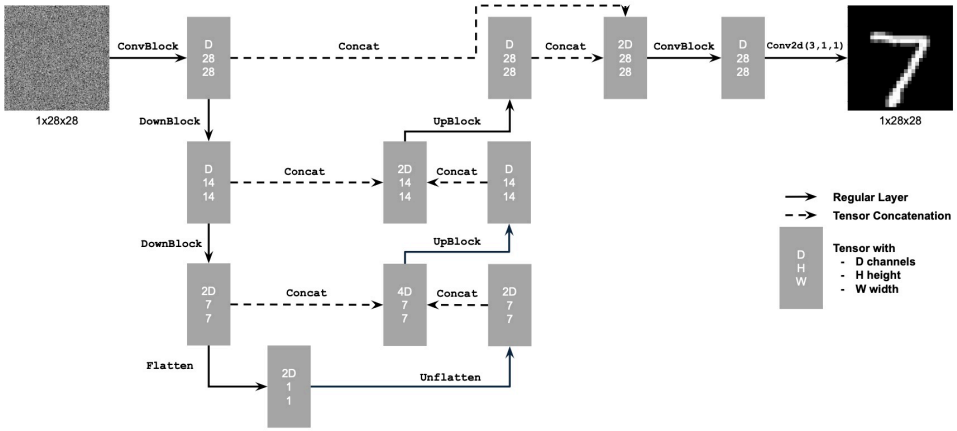
Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

Part B.1: Training a Single-Step Denoising UNet

1.1 and 1.2: Implementing a UNet and Visualizing the Noising Process

After reaching creative flow-state, I trained my own flow matching model on the MNIST dataset! In this first part, I trained an unconditional UNet (meaning it does not take in any text to guide its diffusion process) to denoise noisy inputs back into clean inputs in a single step.



Architecture for an unconditional UNet

The denoising task uses an L2 reconstruction loss, where D_θ is the denoiser, z is a noisy input, and x is a clean image.

$$L = \mathbb{E}_{z,x} \|D_\theta(z) - x\|^2$$

To train the denoiser, I needed to generate training pairs of (z, x) where x is a clean MNIST sample and z is generated from x using the forward noising process:

$$z = x + \sigma\epsilon, \quad \epsilon \sim N(0, I)$$

Here are examples of the noising process over different noise strengths σ = [0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0]. A noise strength of 0 preserves the original clean image, and a higher noise strength results in a noisier input.

Contents

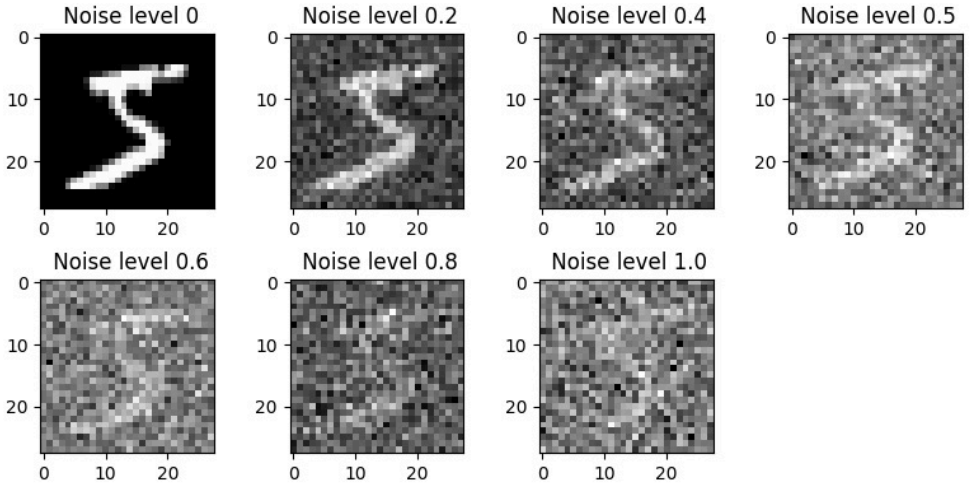
Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model



Adding noise to an MNIST sample with different noise strengths

1.2.1: Training a UNet to Perform Single-Step Denoising

Next, I trained a UNet to denoise a noisy image z, produced by applying σ = 0.5 to a clean image x. I trained the UNet with hidden dimension D=128 for 5 epochs using an Adam optimizer (learning rate of 1e-4).

Here is a plot of the UNet training loss over time, and the results of the UNet denoising samples from the test set afret the 1st epoch of training and after the final 5th epoch of training. Even after 1 epoch of training, the UNet's denoised result already resembles the original inputs quite closely! Training for additional epochs does yield some improvements: the denoised results after 5 epochs of training have clearer, more "confident" edges.

Contents

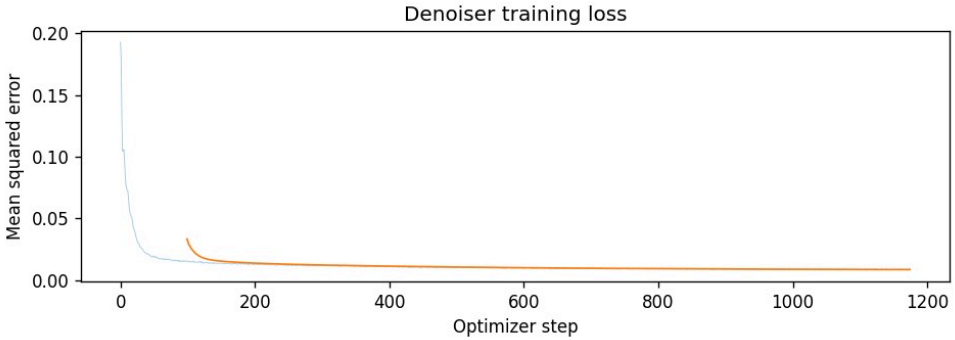
Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

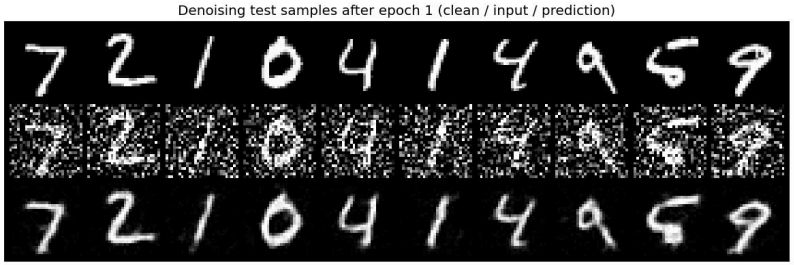
Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

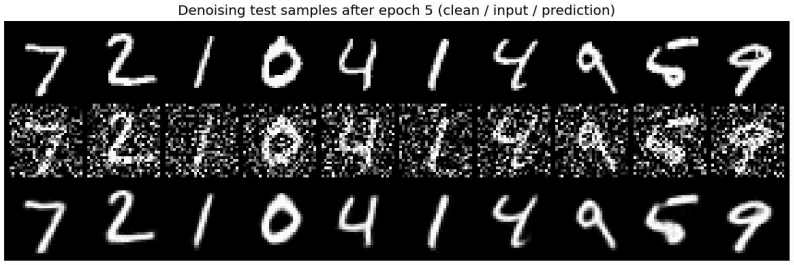
Part B.2: Training a Flow Matching Model



One-step UNet denoiser training loss



Denoising noisy images with σ = 0.5 after 1 epoch of training



Denoising noisy images with σ = 0.5 after 5 epochs of training

1.2.2: Out-of-Distribution Testing

This denoiser was only trained on inputs z with σ = 0.5. I tested the denoiser on different noise strength levels, which are out of its training distribution.

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

Here are the results of the denoiser trying to denoise the same clean image, but with inputs created with varying levels of noise: σ = [0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0]. The higher the noise level, the worse the denoiser was at recovering the original clean image. By contrast, even though the denoiser was not trained on low noise levels like 0 (no noise added at all), 0.2, and 0.4, because the input z still retained much of the original clean image's information, the UNet was still able to recover the original.

Contents

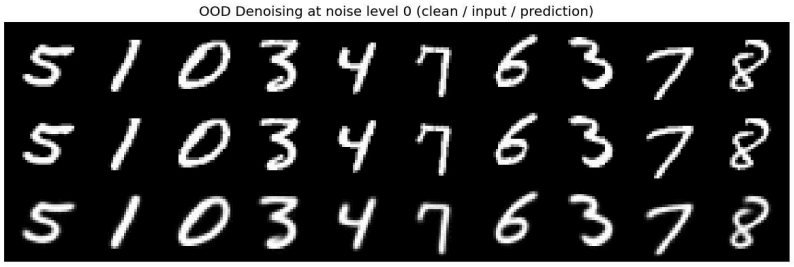
Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

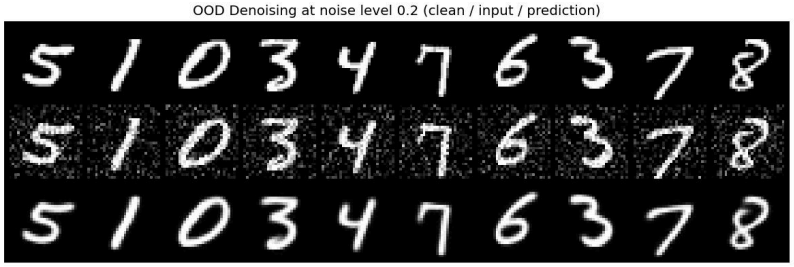
Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

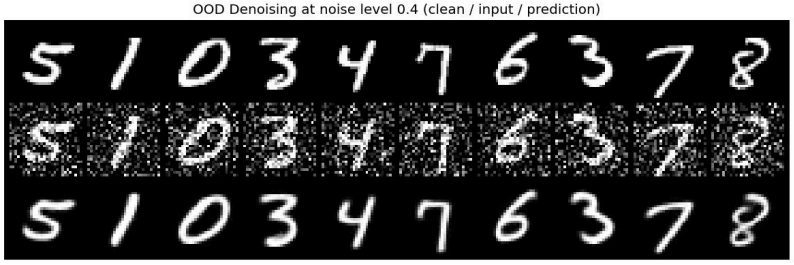
Part B.2: Training a Flow Matching Model



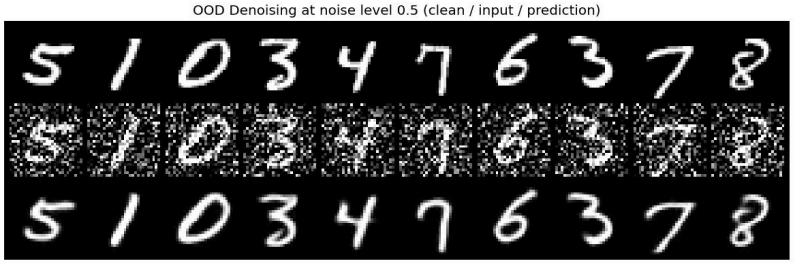
Out-of-distribution denoising at σ = 0



Out-of-distribution denoising at σ = 0.2



Out-of-distribution denoising at σ = 0.4



Contents

Part A.1: Visualizing Training and Sampling

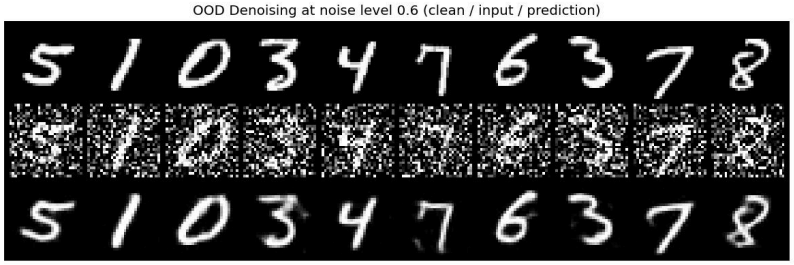
Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

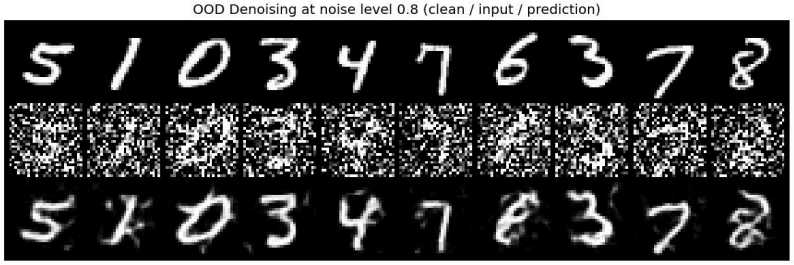
Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

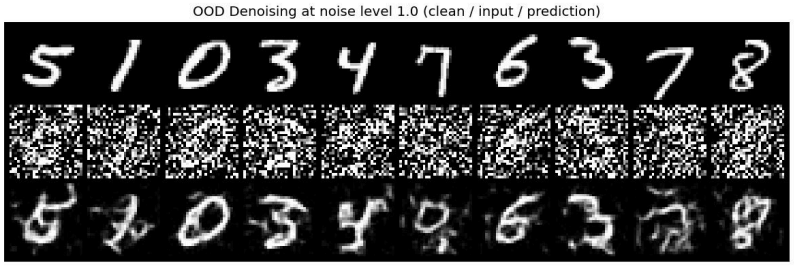
Denoising noisy images at the noise strength level it was trainged on (σ = 0.5)



Out-of-distribution denoising at σ = 0.6



Out-of-distribution denoising at σ = 0.8



Out-of-distribution denoising at σ = 1.0

1.2.3: Denoising Pure Noise

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

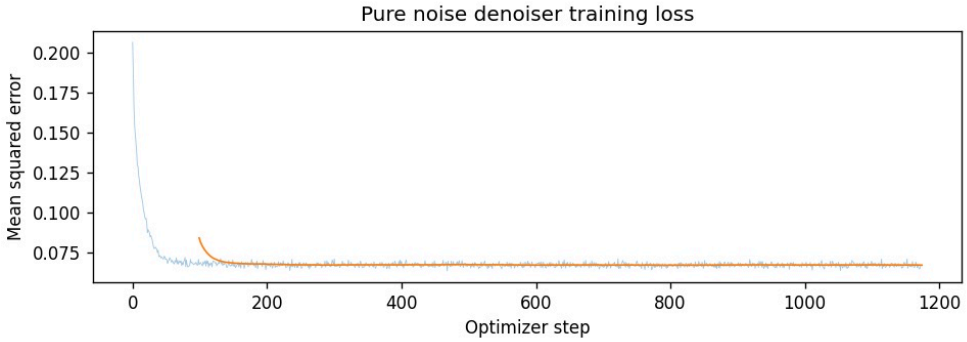
Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

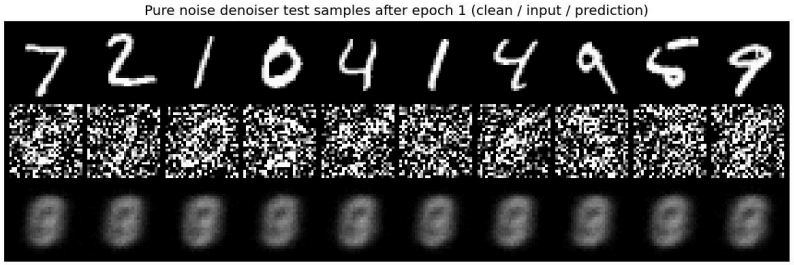
Part B.2: Training a Flow Matching Model

To generate new images from scratch, the model needs to denoise pure Gaussian noise. I trained a new UNet to denoise starting from pure random noise (e.g. z = epsilon, where epsilon is samples from N(0, I)) in order to recover a clean image x.

These are the results of its training loss and denoised samples from the UNet after 1 epoch and 5 epochs of training. The sampled results of the denoiser are a blurry "average" of all the digits from 0-9, regardless of what the initially sampled x was! This makes sense because the input to the denoiser is pure noise, which contains no information at all about what the clean x was. Additionally, the denoiser has no guidance to steer it towards generating one digit or another. Since the denoiser was trained with MSE loss, and MSE loss is minimized by the mean of the population, the denoiser learns to simply output the average digit: all the digits blurred together. Training the denoiser for more epochs has no effect on the blurry average digit it outputs.



One-step pure-noise denoiser training loss



Contents

Part A.1: Visualizing Training and Sampling

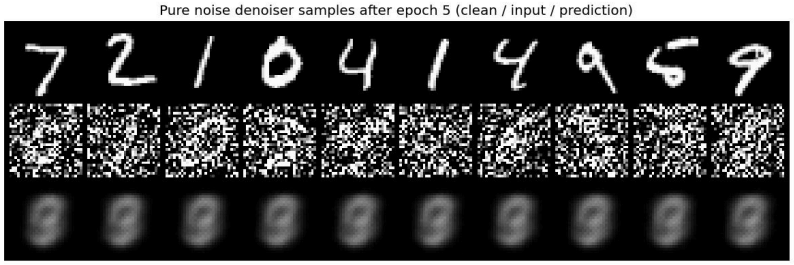
Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

Denoising pure noise after 1 epoch of training: a blurry average digit!



Denoising pure noise after 5 epochs of training: a blurry average digit!

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

Part B.2: Training a Flow Matching Model

2.1: Adding Time Conditioning to the UNet

One-step denoising is not sufficient for generative tasks. To improve on this, we need a model that can iteratively denoise an image in incremental steps. A flow matching model is a model that predicts the flow from noisy data to clean data, AKA the velocity at a given input x_t and timestep t that points towards a real, clean image x_1.

Then, to generate a new image from scratch, we can sample pure noise x_0 ~ N(0, I) and run the backwards flow process to generate a new, realistic x_1.

To train a flow matching model, we can construct intermediate noisy samples by linearly interpolating between x_0 (pure noise) and a clean image x_1 samples from our dataset.

$$x_t = (1-t) \cdot x_0 + t \cdot x_1 \quad \text{where} \quad x_0 \sim N(0, I), t \in [0, 1]$$

By sampling x_t for all possible timesteps t and all possible clean images x_1, we have a vector field that describes the position of a point x_t at time t relative to p_1(x_1), the distribution of clean data, and p_0(x_0), the distribution of noisy data. Flow is akin to the velocity, AKA change in position with respect to time, of this vector field that describes how to move from x_0 to x_1.

$$u(x_t, t) = \frac{d}{dt} x_t = x_1 - x_0$$

The goal of the UNet is thus to learn the flow. The ground truth flow is x_1 - x_0, and the UNet's predicted flow is u_θ

$$L = \mathbb{E}_{x_0 \sim p_0(x_0), x_1 \sim p_1(x_1), t \sim U[0,1]} \left\| (x_1 - x_0) - u_\theta(x_t, t) \right\|^2$$

One way to pass the time-conditioning t into the UNet is to use a fully-connected layer that transforms the scalar t (with shape (1,)) into a multi-dimensional tensor (with shape (D, 1, 1)) that can then be concatenated to other intermediate activations inside the UNet.

Contents

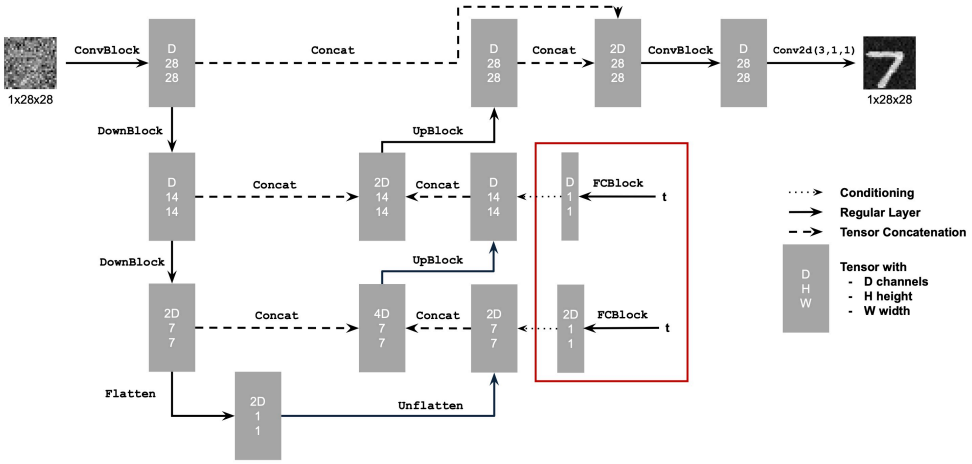
Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

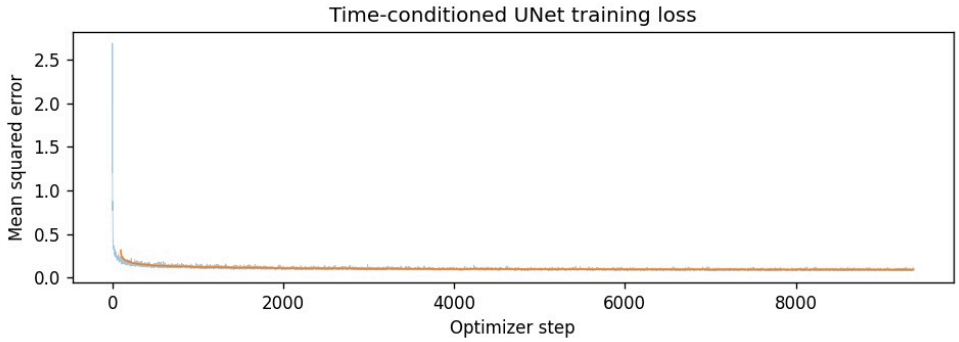


Architecture for a time-conditioned UNet

2.2: Training the Time-Conditioned UNet

When training the time-conditioned flow model, I sampled a random image x_1 from the training set, a random timestep from the uniform distribution [0, 1], linearly interpolated x_1 and pure noise to get an x_t, and compared the denoiser's predicted velocity with the ground truth velocity. After repeating this for many training samples and many timesteps, we now have a time-conditioned denoiser!

I trained my time-conditioned flow model with hidden dimension D=64 for 10 epochs with an Adam optimizer and exponential learning rate decay scheduler. I scheduled the learning rate to start at 1e-2, then exponential decay with a gamma of 0.1^(1/10).



Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

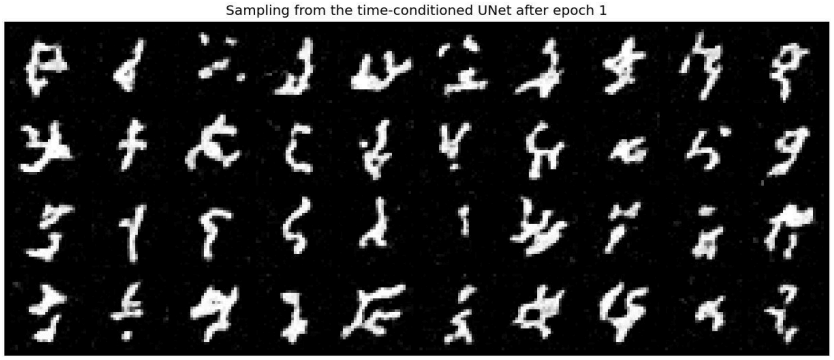
Part B.2: Training a Flow Matching Model

Training loss over 10 epochs for the time-conditioned UNet

2.3: Sampling from the Time-Conditioned UNet

To sample from a time-conditioned flow model, we initialize x_t as pure noise, then take steps from 0 to 1. At each timestep t, the time-conditioned denoiser takes in the current x_t and the current timestep t, then predicts a velocity. Then, we updated the current x_t by taking a tiny step in the direction of the predicted velocity.

I sampled starting with pure noise from the time-conditioned UNet after 1, 5, and 10 epochs of training. The sampled results look more and more digit-like with more epochs of training, but still have some rough edges :) I can pick out the digits 5, 6, 7, and 9 from the model's generated samples, but many others still look like nebulous blobs.



Sampling 40 images from the time-conditioned UNet after 1 epoch of training

Contents

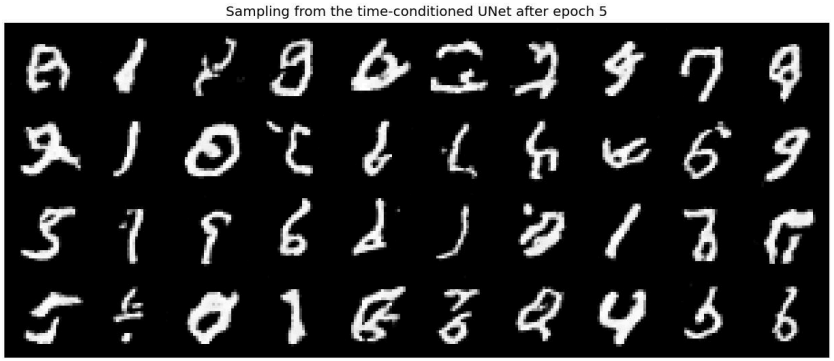
Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

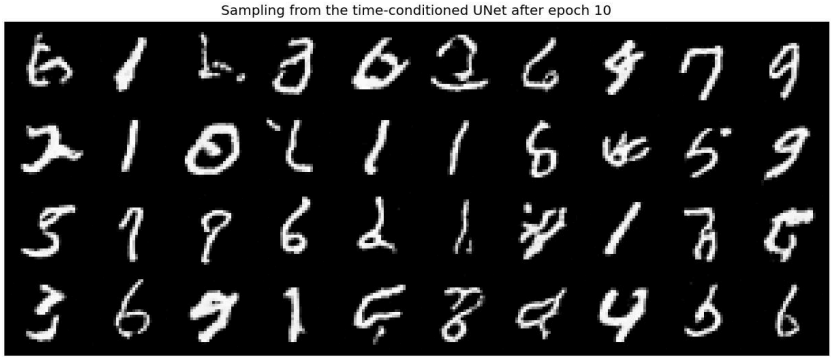
Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model



Sampling 40 images from the time-conditioned UNet after 5 epochs of training



Sampling 40 images from the time-conditioned UNet after 10 epochs of training

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

2.4: Adding Class-Conditioning to the UNet

To further improve the flow model's results and give us more control over what images it generates, we can also add class conditioning: a label 0-9 that steers the model.

I implemeted class-conditioning to my flow model by turning the class into a one-hot vector of dimension 10, where the component corresponding to the requested class is set to 1 and all other components are set to 0. To incorporate class-conditioning to the UNet, I added additional fully-connected layers and concatenated them to the

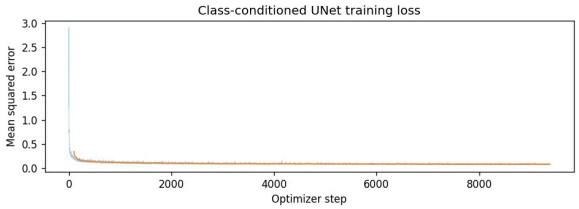
decoder/up-sampling layers.

To support classifier-free guidance, the model also has to be trained to generate images without any class-conditioning. This corresponds to a class vector of all zero's. So during training, I zero-ed out the class-conditioning vector with dropout probability p_uncond.

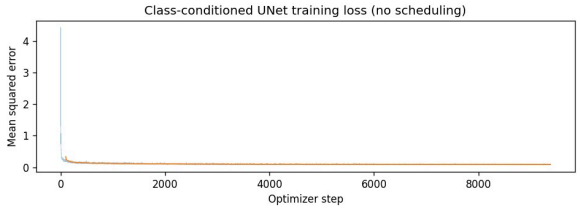
2.5: Training the Class-Conditioned UNet

During training, I sample x_1 and its class c from the training set, a random timestep t from the uniform distribution [0, 1], pure Gaussian noise, and interpolate x_1 and pure noise to get x_t. With drop-out probability p_uncond, I set the class conditioning vector c to be the zero vector. Then, I pass x_t, t, and c to the flow model. Lastly, I compute the loss between the model's predicted velocity and the ground truth velocity, take a gradient update step, and repeat!

I trained the class-conditioned flow model with the same parameters as the time-conditioned flow model (D=64, 10 epochs, Adam optimizer with intiial learning rate 1e-2). I experimenting training the flow model *with* the exponential learning rate scheduler, and *without* the exponential learning rate schduler.



Training loss of the class-conditioned flow model with an exponential learning rate scheduler



Training loss of the class-conditioned flow model without a learning rate scheduler

2.6: Sampling from the Class-Conditioned UNet

Here are the results of sampling from the class-conditioned flow model after 1, 5, and 10 epochs of training. The results on the left are from training with a scheduler, and the results on the right are from training without an exponential learning rate

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

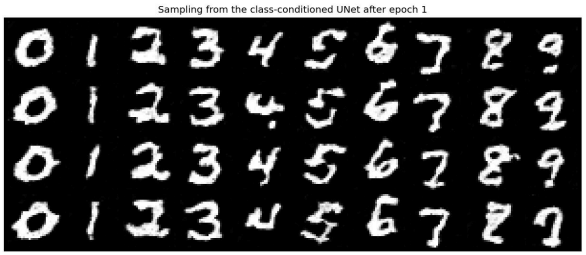
Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model

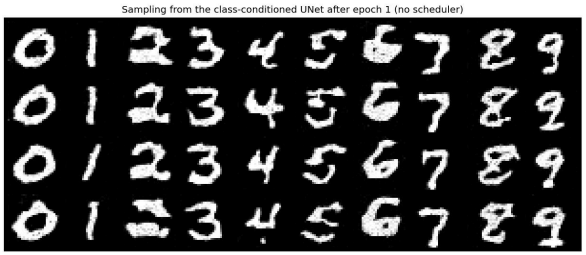
scheduler.

After each epoch, I generated 40 images and evenly iterated through the classes 0-9 (4 instances per class). During sampling, I ran 2 forward passes through the model: one pass with the class vector to predict a conditional velocity, and another pass with a zero vector to predict an unconditional velocity. Then, I combined then using classifier-free guidance with a guidance scale of 5.0.

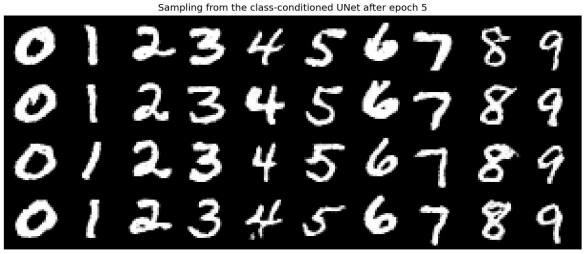
For both training runs (with and without a scheduler), the results improve from epoch to epoch. In the early epochs, training with or without a scheduler yield similar sample results. In the later epochs, I noticed small differences between the two training runs: the model trained with a scheduler generates digits with cleaner edges and more consistent thickness, whereas the model trained without a scheduler generates digits that look slightly shake-y or blob-y – but overall, the models trained in both runs can produce images from scratch that visually pass for MNIST digits! In order to compensate for the lack of a learning rate scheduler while maintaining similar performance, I used a smaller learning rate.

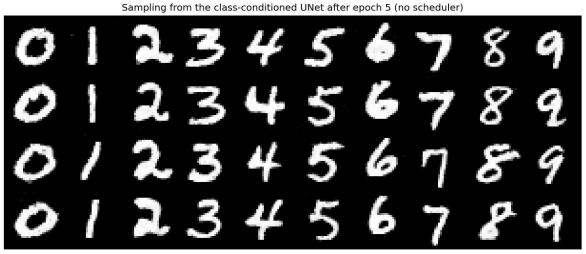


Sampling images from pure noise with a class-conditioned flow model after 1 epoch of training (with scheduler)



Sampling images from pure noise with a class-conditioned flow model after 1 epoch of training (no scheduler)





Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

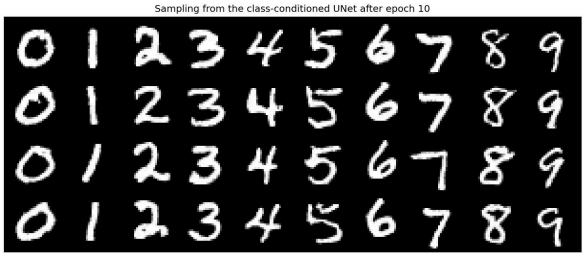
Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

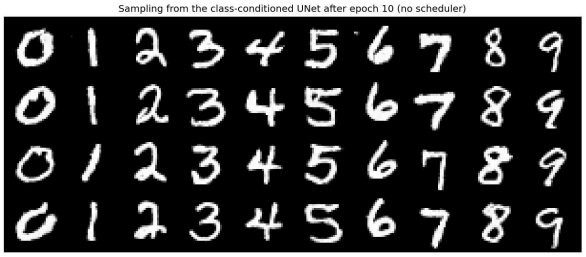
Part B.2: Training a Flow Matching Model

Sampling images from pure noise with a class-conditioned flow model after 5 epochs of training (with scheduler)

Sampling images from pure noise with a class-conditioned flow model after 5 epochs of training (no scheduler)



Sampling images from pure noise with a class-conditioned flow model after 10 epochs of training (with scheduler)



Sampling images from pure noise with a class-conditioned flow model after 10 epochs of training (no scheduler)

Contents

Part A.1: Visualizing Training and Sampling

Part A.2: Conditional Generation and Guidance

Part A.3: Creative Applications

Part B.1: Training a Single-Step Denoising UNet

Part B.2: Training a Flow Matching Model